

MSDM 5001

Introduction to Computational and Modeling Tools

Lecture 1: Course introduction, the Linux OS and the terminal

Logistics

- 9:30 – 12:20 every Saturday 🙄
- Check Canvas for materials and announcements!
- Teaching staff:
 - Me
Office: 4433
Email: phycsan@ust.hk
 - IA: ZHU, Jundong
Office: 4469
Email: zhujd@ust.hk

Office hours:
By appointment

Grading structure

- Assignment x4: 60%
- In-class quiz x2: 35%
- Attendance/participation: 5%

Course schedule



Weeks	
1	Setting up Linux and Python environments, intro to Linux OS, basic terminal commands, bash scripting
2	Version control with Git and GitHub
3	Relational database management, SQL
4	Basic computer architecture (von Neumann architecture and variations, modern CPUs and GPUs)
5	Intro to HPC, concepts in parallel computing (Amdahl's law, thread vs process, shared vs distributed memory, communication, synchronization, load balancing etc.), common patterns and pitfalls of parallel program designs, performance profiling
6	Parallel computing in Python I (threading, multiprocessing, mpi4py etc.)
7	Parallel computing in Python II (threading, multiprocessing, mpi4py etc.)
8	Python for scientific computing I (numpy, scipy, numba, scikit-learn, Cython etc.)
9	Python for scientific computing II (numpy, scipy, numba, scikit-learn, Cython etc.)
10	Intro to quantum computing I (basic computer science theory, basic quantum mechanics)
11	Intro to quantum computing II (qubits, quantum circuits, real-life implementations)
12	Intro to quantum computing III (quantum algorithms: Grover, Shor etc.)
13	Spare lecture. Topic TBD (if time permits)

Quiz

Assignment

Policies

- Academic Integrity:

<https://registry.hkust.edu.hk/resource-library/academic-integrity>

- Use of generative AI:

You are allowed to use AI-tools to assist you in learning.

HOWEVER, you must ensure all final works reflect your own understanding and analysis.

Direct copying of AI-generated content without attribution constitutes **academic dishonesty**, and you remain fully responsible for the accuracy and originality of all submitted work.

Some disclaimers and advice

- This is NOT a course on hardware! But we do cover hardware concepts at a high level.
- We cover a diverse range of topics, so we don't have a dedicated textbook. Following along lecture slides and notes should suffice. I will also list some useful references.
- Feedback is welcome!

What you should know

- Intermediate Python
 - What's the difference between a list and a tuple? When would you use each?
 - What is a generator in Python? Examples? Use cases?
 - What is a decorator in Python? Examples? Use cases?
 - How is a Python list different from a C-style array and a C++ vector?
 - Can you use lists as keys in a dictionary?
 - What does it mean for Python to be an **interpreted** language?
 - What makes Python "slow"? **

What you should know

- Basic computer science
 - Name some sorting algorithms, and compare their advantages vs disadvantages
 - What is a hash-map?
 - Basic big-O complexity
- Basic linear algebra
 - What is a vector space? What is a basis of a vector space? What is an inner product on a vector space?
 - What is a linear operator/matrix?
 - Basic properties of matrix: unitary, symmetric, positive-definite etc.
 - What are eigenvectors and eigenspaces of a matrix? How do you compute them?

Any questions?

Before you proceed...

- Have you set up Linux environment on your computer? *

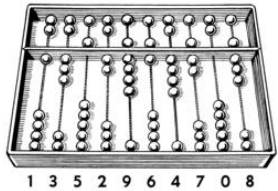


* If you use a Mac, you *should* be fine, but not everything we covered today will be supported

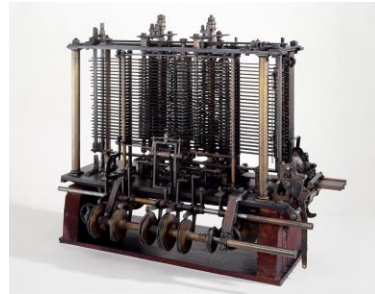
what IS a computer? 🤔

A brief history of computers

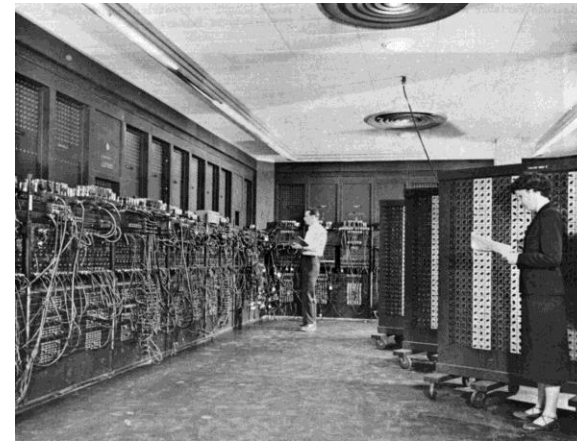
Abacus
ca. 2400 B.C.



Analytical Engine
1837



ENIAC
1946



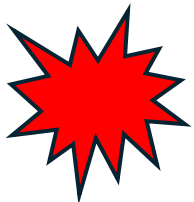
Antikythera mechanism
ca. 200 B.C.



Turing Machine
1936

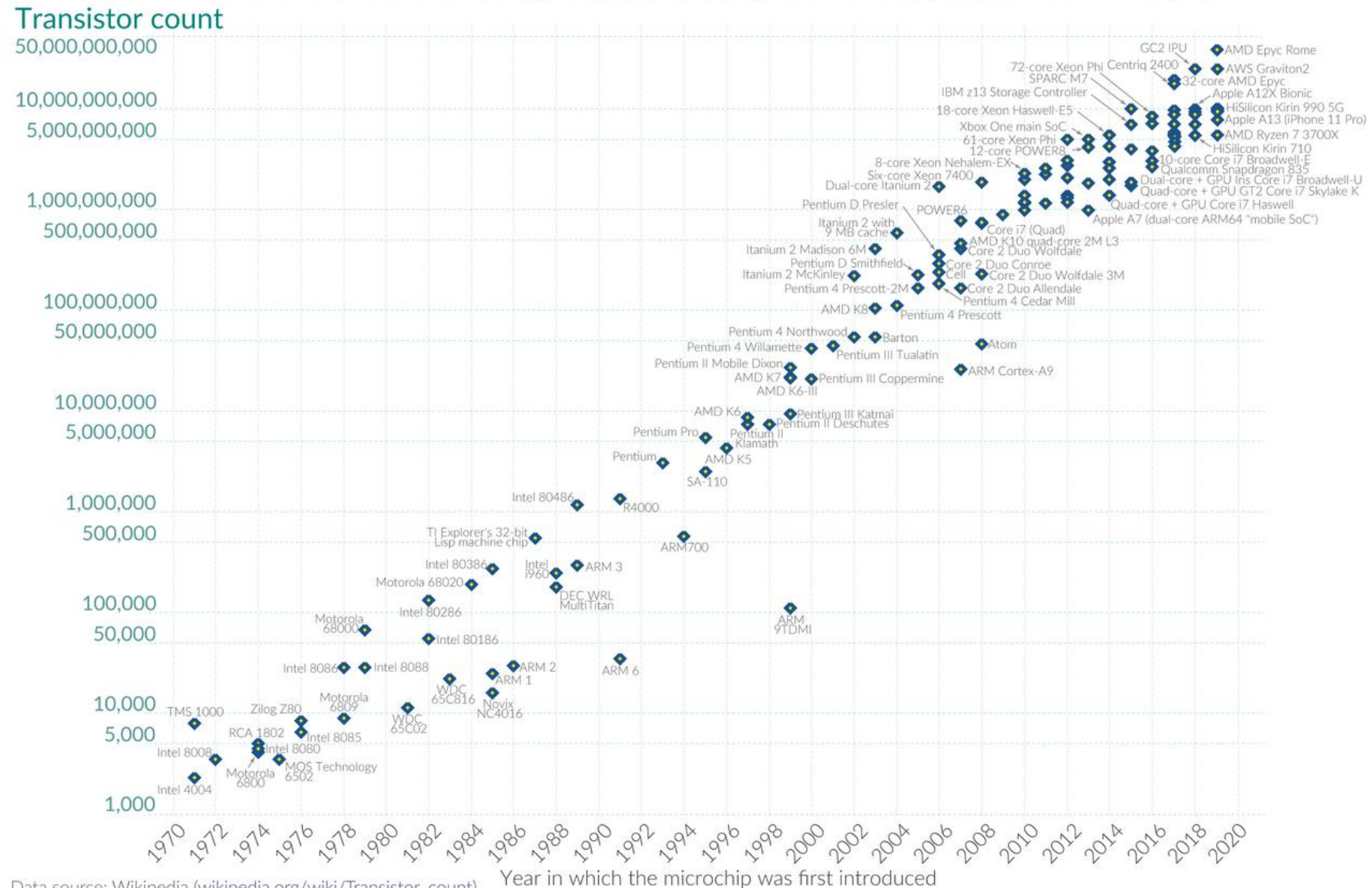


Transistors
1947



Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Supercomputers...



TOP500 LIST - JUNE 2025

R_{max} and R_{peak} values are in PFlop/s. For more details about other fields, check the TOP500 description.

R_{peak} values are calculated using the advertised clock rate of the CPU. For the efficiency of the systems you should take into account the Turbo CPU clock rate where it applies.

[←](#)[1-100](#)[101-200](#)[201-300](#)[301-400](#)[401-500](#)[→](#)

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

Rise of personal computers

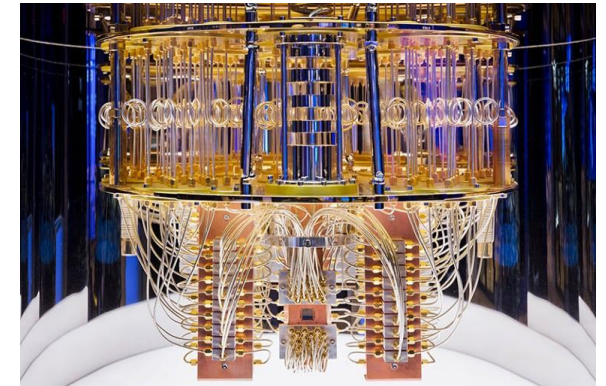
From here...



To here...



What comes next???



How do we harness
power of modern
computers?

Programming Languages

Early 40s: Machine code

1949: Assembly language

1957: FORTRAN

1959: COBOL

1972: C

1978: SQL

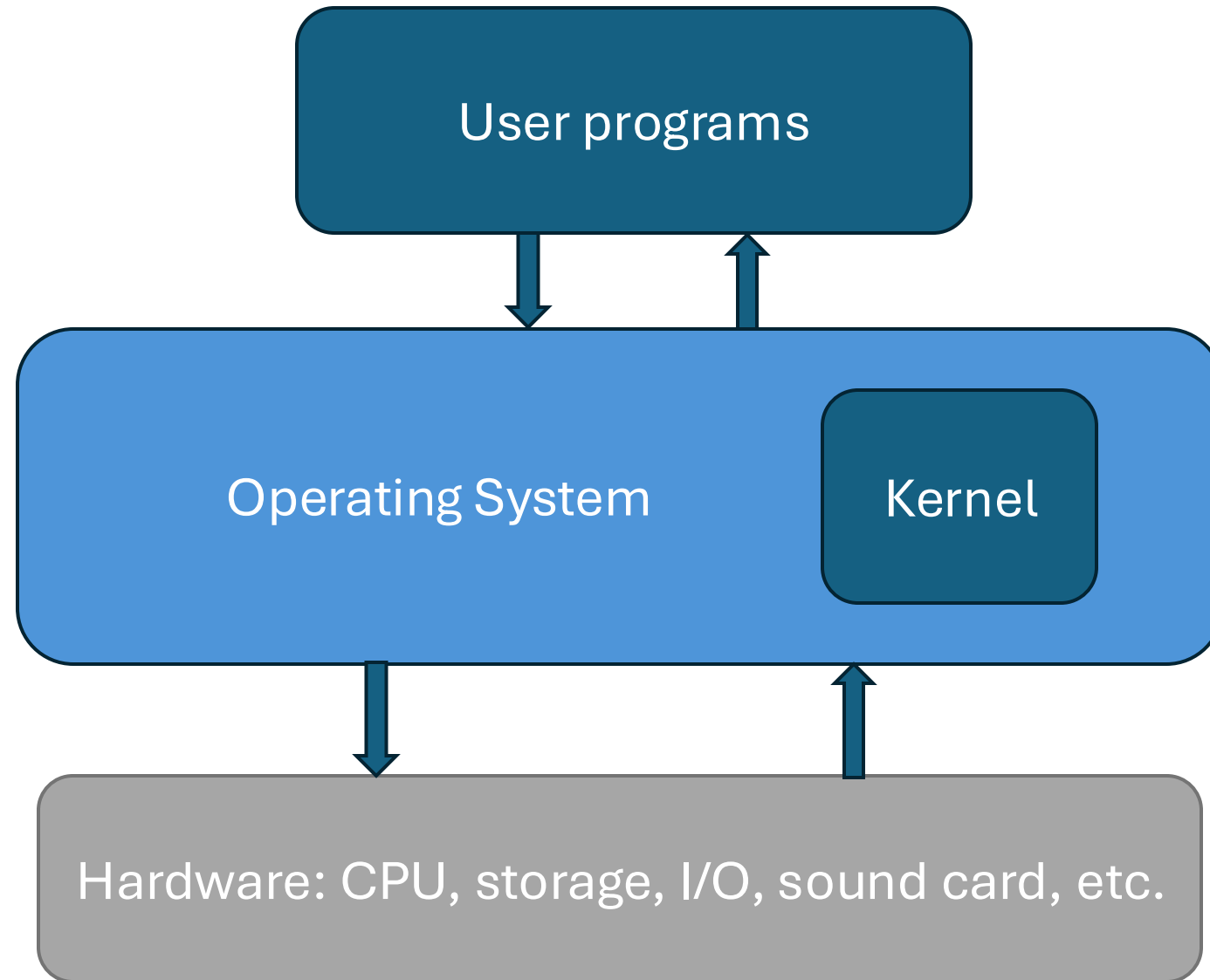
1991: Python

2010: Rust

2012: Julia



Basic Computer Hierarchy





忠告：借錢梗要還 咪俾錢中介

查詢及投訴熱線：3199 1199

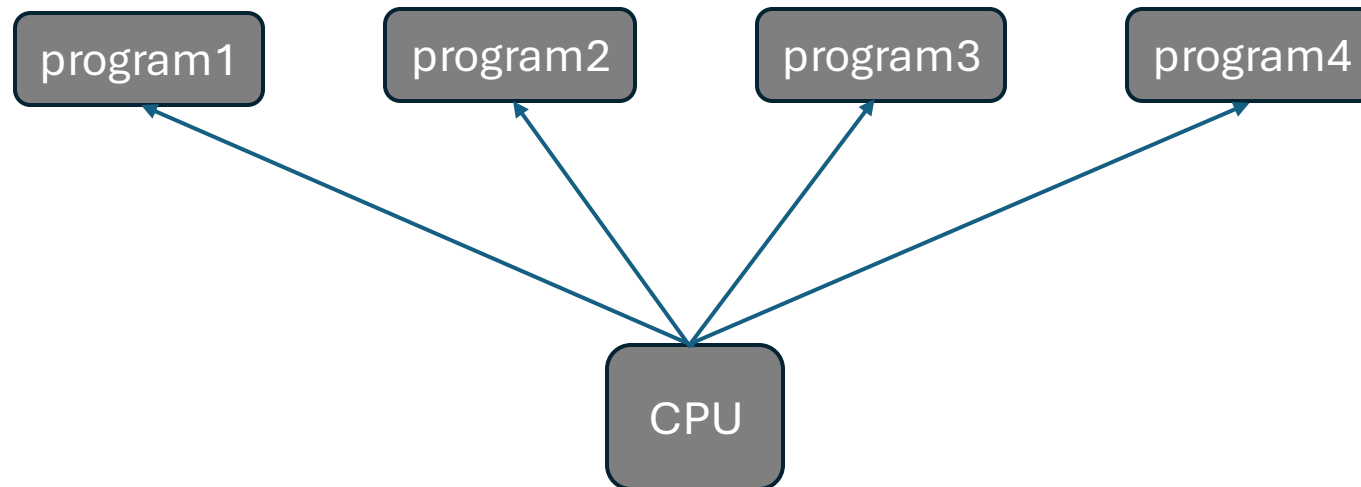
Question: why a middleman??

Jobs of an OS

- Resource allocations
- Ensure security
- Jobs scheduling (crucial to multitasking and multithreading!!)
- And more...

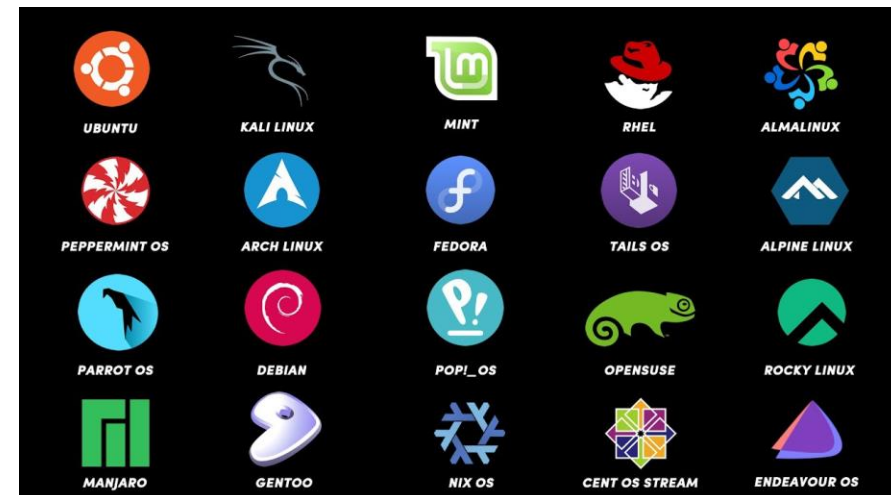
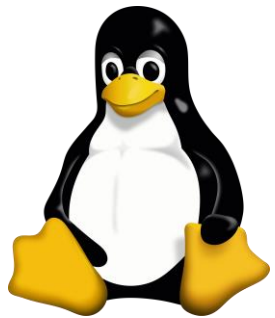
An aside on multitasking

- It is possible even with just ONE CPU!
- An example of **concurrency**



The Linux Operating System

- Open-source operating system **kernel** originally created by Linus Torvalds in 1991 (age 21!)
- Unix-based
- (mostly) POSIX-compliant
- Comes with different flavours ("distros")



Why Linux?

- Optimized for server and high-performance computing
- Stable
- Free **入場無料**
- Highly customizable for specific needs
- Industry standard: Most servers and supercomputers run on Linux
- Teaches you a lot about computers!

This is not a sponsored slide



Linux Filesystem

- Linux uses a "simple" filesystem
 - Hierarchical, tree-like
 - Case-sensitive (unlike Windows or Mac)
 - Starts with the "root" folder/directory: "/"
 - Current directory is "."
 - Parent directory is ".."
 - "home" directory is "~"

```
eric@fedora:~$ tree / -L 1
/
├── afs
├── bin -> usr/bin
├── boot
├── dev
├── etc
├── home
├── image
├── lib -> usr/lib
├── lib64 -> usr/lib64
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin -> usr/sbin
├── srv
├── sys
├── tmp
├── usr
└── var

22 directories, 0 files
```

The Terminal

- A text-based interface to communicate with the operating system
 - Windows: Powershell, CMD
 - Mac & Linux: bash, zsh, fish, ksh, ...
- Can be much more powerful than GUIs!
- On Linux, usually instantiated by Ctrl + Alt + T

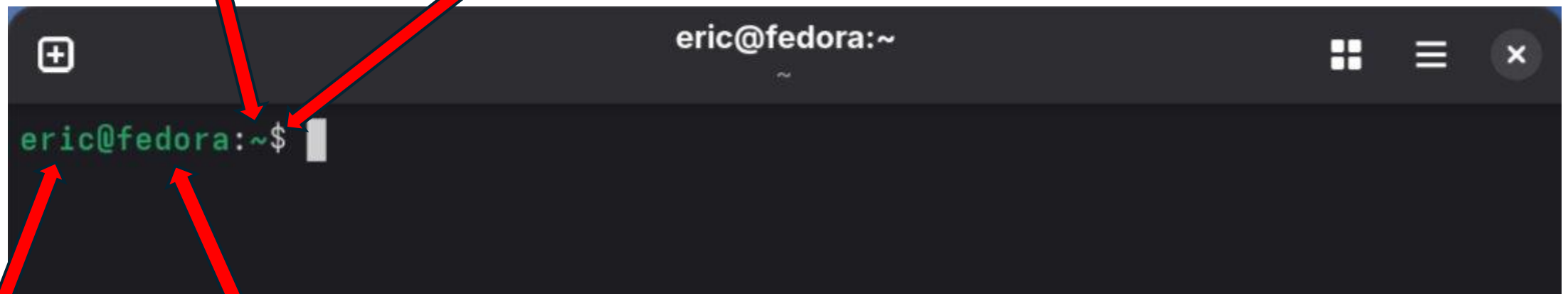
Task: Open the terminal on your computer

current directory

"privilege" level

username

host/computer name



Some basic terminal commands

- man: manuals for various commands
 - sometimes only --help is available, esp. for newer programs
- pwd: print current working directory
- ls: list contents in current directory
- cd: change directory
- sudo: run command with root privileges

Basic file operations

- mkdir: create directory
- cp: copy files/directories
- mv: move files/directories
- rm: remove files/directories
- touch: updates the "last modified" time to now, can create new file
- nano/vi: "basic" text editors



cp and mv will silently OVERWRITE any existing file/directories.
Use the -i flag to be safe!

Question: If you want to rename your file/directory,
which command should you use?

Practice 1



1. If you are not at the home directory (~) yet, go there
2. List all contents in your current directory in the long format
3. Create a directory called "Practice" in your home directory
4. Go to this new directory
5. Create two empty text files called "hello.txt" and "hello2.txt"
6. Open "hello.txt" with nano, type something, and save it
7. Rename "hello2.txt" to "world.txt"
8. Create a new directory called "Documents" inside the "Practice" directory, and move the two text files inside
9. Go back to your home directory, and delete the Practice directory (and everything inside)

Practice 2



1. Check your version of Python using "python --version"
2. Find the location of the Python interpreter using "which python"
3. Create an empty Python script called "hello.py"
4. Use a text editor of your choice to write a simple Python program that (you can leave the terminal at this step if you want)
 - Asks the user to input their name
 - Prints "Hello, <name_of_user>!"
 - Repeats above in an infinite loop unless user types in "stop"
5. Run the script from the terminal by typing "python hello.py"

A quick intro to glob and regex

- Both are used to perform pattern matching
 - glob: for matching **file names** and **path names**, used primarily in shell
 - regex: for **matching strings in texts**, used in various programs, much more powerful

patterns	glob	regex	Explanation/examples
Any single character	?	.	
Zero or more preceding character	N/A	*	"a*" can match with "", or "aaa"
One or more preceding character	N/A	+	"a+" can match with "aaa", but not ""
Zero or one preceding character	N/A	?	"a?" Can match with "" or "a", but not "aa"
Any string (even empty)	*	.*	
A literal dot	.	\.	"\" is an "escape character"

Example:

In regex, "**ab*c**" can be matched with "ac", "**abc**", "**abbc**", but not "abhelloc"

In glob, "**ab*c**" can be matched with "**abhelloc**"

A quick intro to glob and regex

patterns	glob	regex	Explanation/examples
Character class	[abc]	[abc]	"a", "b", or "c"
Character range	[a-z]	[a-z]	Any lowercase letter
Negated character class	[!abc]	[^abc]	Any character except "a", "b", and "c"
Repetition of preceding character	N/A	{m,n}	"a{2,4}" matches with "aa", "aaa", "aaaa", and "baaa"
Start of a line/string	N/A	^	"^a{2,4}" matches with "aa", "aaabc", but not "bcaaa"
End of a line/string	N/A	\$	

More examples

1. If you want to match "dog", "Dog", "dogs" or "Dogs":

- regex: "[dD]ogs?"

- glob: no exact parellel.

"[dD]og*" could work, but it also matches with "doge"



2. If you want to match "color" or "colour":

- regex: "colou?r"

- glob: no exact parallel, can only resort to "*", not ideal.

Finding files



- Two ways
 - `ls <glob_pattern>`:
lists all files/directories matching `<glob_pattern>` in current directory
 - `find`:
much more powerful than `ls`

`find [path] [expressions]`

Expressions are typically:
`[tests][action]`

```
1 $ find . -name "*.txt" # Find all .txt files in
    current directory
2 $ find /home -name "config" # Find files/dirs named "
    config" in /home
3 $ find . -type f -name "*.py" # Find only files (not
    directories) with .py extension
4 $ find . -type d -name "test*" # Find only directories
    starting with "test"
5 $ find . -size +100M # Find files larger than 100
    MB
6 $ find . -mtime -7 # Find files modified in last
    7 days
7 $ find . -user username # Find files owned by
    specific user
```

Advanced Examples

```
1 $ find . -name "*.log" -delete # Find and delete all .log
    files
2 $ find . -name "*.txt" -exec cat {} \; # Find .txt files and
    display contents
3 $ find /var/log -name "*.log" -size +10M # Find large log files
```


Finding patterns/strings in files



- grep: uses regex to match patterns inside files
 - prints all matched substring by default

```
grep [options] <patterns> [file(s)]
```

- Examples:
 - grep "[A-Z]" file.txt:
print all lines that start with a capital letter
 - grep -r "password" .
searches for all lines containing "password" in all files inside the current directory (including sub-directories)
 - grep -l "main" *.py
prints all **file names** that end with .py inside the current directory and contain "main"



Practice 3

1. Find all files ending with **".txt"** under the root directory larger than 1 MB. You might need root privilege with sudo
 - You will notice that find will search through all subdirectories recursively, how do we limit the search to only, say, 2 levels beneath the root?
2. Type in the following to the terminal and hit Enter:
`curl -o shakespeare.txt "https://www.gutenberg.org/files/100/100-0.txt"`
 - Count how many lines in shakespeare.txt contains the word "twas"
 - How about "Twas"?
 - How about number of lines that include either "twas" or "Twas"?
 - Do the numbers above add up? Why?
3. Count how many files under the home directory contains the word "import" in them.

Redirection operators

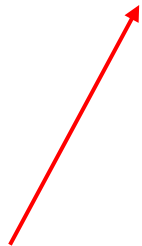
- Sometimes we don't want the output of a command to be shown on the terminal directly
 - `>` : Redirect output to file (overwrite if exists)
 - `>>` : Redirect output to file (append if exists)
 - `2>` : Redirect error output to file (overwrite if exists)
 - `2>>` : Redirect error output to file (append if exists)
- Example
 - `find . -name "*.py" > python_programs.txt`



Pipe operator |

- Alternatively, we may want to use output of a command as **standard input** of another. This is when we use |
- Examples:
 - `ls /bin /usr/bin | sort | less`
 - `ls /bin /usr/bin | sort | uniq | less`
 - `find /home -name ".mp3" | xargs du -ch | tail -1`

Turns stdin into
command arguments



Practice 4



1. Save the full paths of all files under / that end with ".c" into a file titled "c_programs.txt"
(you can suppress permission errors by 2>/dev/null)
2. Find the 5 largest files under /usr/bin
You may need to use du, sort, and head/tail
3. Count how many files end with ".py" under /
You may need root privilege with sudo.
You may need to use wc.

A seemingly useless command: echo

- echo prints whatever you give it back to the standard output. Kind of like the print function.

```
eric@fedora:~$ echo "Hello World!"  
Hello World!  
eric@fedora:~$ █
```

- But this? 🤔

```
eric@fedora:~$ echo $USER  
eric  
eric@fedora:~$ echo /*  
/afs /bin /boot /dev /etc /home /image /lib /lib64 /media /mnt /opt /proc /root /run /sbin /srv /sys /tmp /usr /var  
eric@fedora:~$ █
```

Variables in the shell

- Environment variables
 - Available to all processes and sub-shells
 - Accessed via `env` or `printenv`
 - e.g. `HOME`, `USER`, `PATH`, ...
- Shell variables
 - Local to the current shell session
 - Declared via `"var=val"` (no spaces!)
 - Can be promoted to environment variables by the `"export"` command

Question: When you type in name of a program, say `grep`, into the shell, how does it know where to look for this program? Does it look through all directories? This seems silly and inefficient...

Shell expansion

- Before commands/programs are run, the shell first automatically processes patterns and variables
 - In `"echo *"`, the asterisk automatically expands to all filenames in pwd
 - In `"echo $USER"`, the dollar sign extracts values of the USER environment variable
- Different kinds of expansions
 - Pathname expansion: same as globbing before
 - Command substitution: `echo "Today is $(date)"`
 - Brace expansions:
 - `{a,b,c}` # → a b c
 - `file{1..5}.txt` # → file1.txt file2.txt file3.txt file4.txt file5.txt
 - `{01..10}` # → 01 02 03 ... 10 (zero-padded)
 - `{a..z..2}` # → a c e g ... (step by 2)

Practice 5



1. Create directories titled "Lecture_01", ..., "Lecture_13" in one line
2. Examine the PATH environment variable, where does it point to?
3. Create a timestamped directory and copy all files ending with ".txt" into it (this is the most naïve way to back up your files)
Hint: look into the "date" command

Shell Scripting

- Program to execute a sequence of terminal commands
- Can define variables, have control structures, loops etc.
- Kind of archaic syntax, so use it mostly for simple tasks
Use Python for more complicated tasks

"shebang":
specifies which
shell to use

hello.sh



```
1 #!/bin/bash
2
3 echo "Hello World!"
```

To run this script:
chmod +x ./hello.sh # make it executable
./hello.sh

If-else statements

```
if [ condition ]; then
    # commands if condition is true
elif [ another_condition ]; then
    # commands if another_condition is true
else
    # commands if all conditions are false
fi
```

- Examples of conditions
 - Numeric comparisons: -gt, -lt, -le, -ge, -ne, -eq
 - String comparisons: =, !=, \>, \<, -z, -n
 - File tests: -e, -f, -d
 - Logical operators: -a, -o, !

For loops

```
for variable in list_of_items; do
    # commands using $variable
done
```

- Examples of list_of_items:
 - Explicit list: Alice Bob Charlie, 1 2 3 4 5
 - Brace expansion: {1..12}, {01..12}, {a..z}

```
1  #!/bin/bash
2
3  mkdir album
4  cd ./album
5
6  for year in {15..25}; do
7      for month in {01..12}; do
8          mkdir "photo_${month}_20${year}"
9      done
10 done
```

While loops

```
while [ condition ]; do  
    # commands  
done
```

```
1 #!/bin/sh  
2  
3 counter=3  
4  
5 while [ $counter -ge 0 ]; do  
6     echo "${counter}..."  
7     counter=$((counter - 1))  
8     # ((counter -= 1))  
9 done  
10  
11 echo "BOOM!"
```

Shell script arguments

- Bash automatically provides command-line arguments through special variables:
 - \$0: name of script
 - \$1: first argument
 - \${10}: 10th argument (braces needed) 传给脚本的第10个参数 (注意10及以上的参数需要加花括号)
 - \$#: number of arguments
 - "\$@" # All arguments as separate quoted words

Practice 6

- You have a bunch of photos with names like IMG_001.jpg, IMG_002.jpg, etc., and you want to organize them by batch into folders. Write a script that
 - Takes a batch size as an argument (e.g., 10)
 - Groups photos into folders like batch_1, batch_2, etc.
 - Shows progress as it works
- Expected output

```
$ ./organizer.sh 10
Creating batch_1...
Moving IMG_001.jpg to batch_1
Moving IMG_002.jpg to batch_1
...
Moving IMG_010.jpg to batch_1
Creating batch_2...
Moving IMG_011.jpg to batch_2
...
Organized 25 photos into 3 batches
```

Just the start...

- There is so much more to the terminal, but we'll stop here
- Useful references:
 - "The Linux Command Line" by William Shotts
 - "How Linux Works" by Brian Ward
 - For those who are reaaaaaally interested:
<https://www.linuxfromscratch.org/lfs/view/stable/>

Before next lecture

- Please sign up for a GitHub account, if you don't have one already
- Make sure you have git installed